



Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering (PCCoE)

(An Autonomous Institute)

Affiliated to Savitribai Phule Pune University (SPPU)

ISO 21001:2018 Certified by TUV SUD

TY B.Tech

Year:2025-26

Semester:6

Name: Paritosh Nimhan

PRN:123B1B212

Branch: Computer Engineering

Div:C

Assignment 7

Title: Design a web application for e-commerce or students' feedback review systems or any other applications with a MongoDB Backend using React.js and Express.

Objectives:

- To design and develop a web-based application that allows students to submit feedback about courses and learning experience.
- To provide an easy interface for users to add, view, update, and delete feedback
- To store and manage feedback records using a NoSQL database (MongoDB).

Outcomes:

- Understanding of frontend development using React.js and component-based architecture
- Experience in working with MongoDB for storing and retrieving feedback data.
- Implementation of CRUD operations (Create, Read, Update, Delete).
- Development of a functional feedback system connecting frontend and backend.

Theory Concept:

Introduction

The Student Feedback System is a full stack web application developed to collect, manage, and analyze feedback provided by students for different courses. It is built using React.js for the frontend, Express.js for the backend, and MongoDB for database management.

The system follows a client-server architecture, enabling efficient communication between users and the server for performing various operations.

System Overview

The Student Feedback System is designed to simplify the process of collecting and managing feedback.

Main Functionalities:

- Submit feedback for courses
- View all feedback entries
- Edit previously submitted feedback
- Delete feedback records

The system provides a user-friendly interface and ensures efficient data handling.

Feedback Management System

The system digitizes the traditional feedback collection process and offers several benefits:

- Eliminates manual paper-based feedback
- Stores feedback securely in digital format
- Allows easy modification and deletion of records
- Provides quick access to feedback data
- Helps institutions analyze student opinions

This leads to improved decision-making and better course management.

Dataset Used

- No external dataset was used.
- Data is dynamically generated and stored in MongoDB collections during runtime

Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Feedback System</title>
  <!-- React CDNs -->
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin="anonymous"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin="anonymous"></script>
  <script src="https://cdn.jsdelivr.net/npm/regenerator-runtime@0.14.0/runtime.min.js"></script>
  <!-- Babel for JSX -->
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
  <!-- Axios for API calls -->
  <script src="https://unpkg.com/axios/dist/axios.min.js"></script>
```

```
<style>
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: Arial, sans-serif;
}

body {
  background-color: #f9fafb;
  color: #333;
  min-height: 100vh;
  padding: 2rem 1rem;
}

.container {
  max-width: 800px;
  margin: 0 auto;
  background: #ffffff;
  padding: 2rem;
  border: 1px solid #e5e7eb;
  border-radius: 4px;
}

.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 2rem;
  padding-bottom: 1rem;
  border-bottom: 2px solid #2563eb;
}

.header h1 {
  font-size: 1.5rem;
  color: #1e3a8a;
}

.btn {
  padding: 0.5rem 1rem;
  border-radius: 4px;
  border: 1px solid #2563eb;
  cursor: pointer;
  font-size: 0.9rem;
  transition: all 0.2s;
}

.btn-primary {
```

```
    background: #2563eb;
    color: white;
}

.btn-primary:hover {
    background: #1d4ed8;
}

.btn-danger {
    background: white;
    color: #dc2626;
    border: 1px solid #dc2626;
}

.btn-danger:hover {
    background: #dc2626;
    color: white;
}

.btn-outline {
    background: white;
    color: #2563eb;
}

.btn-outline:hover {
    background: #eff6ff;
}

.form-group {
    margin-bottom: 1rem;
}

label {
    display: block;
    margin-bottom: 0.25rem;
    font-weight: bold;
    color: #4b5563;
}

.form-control {
    width: 100%;
    padding: 0.5rem;
    border: 1px solid #d1d5db;
    border-radius: 4px;
    font-size: 1rem;
}

.form-control:focus {
```

```

        outline: none;
        border-color: #2563eb;
    }

    .stars span {
        color: #d1d5db;
        font-size: 1.5rem;
        cursor: pointer;
    }

    .stars span.active {
        color: #2563eb;
    }

    .grid {
        display: grid;
        grid-template-columns: 1fr;
        gap: 1rem;
    }

    .card {
        border: 1px solid #e5e7eb;
        padding: 1rem;
        border-radius: 4px;
        background-color: #f8fafc;
    }

    /* Replacing glass-panel with simple styling */
    .glass-panel {
        background-color: #ffffff;
        border: 1px solid #e5e7eb;
        border-radius: 4px;
        padding: 2rem;
    }

    .card-actions {
        display: flex;
        gap: 0.5rem;
        margin-top: 1rem;
    }
</style>
</head>
<body>
  <div id="root"></div>

  <script type="text/babel">
    const { useState, useEffect } = React;

```

```

// Custom Star Rating Component
const StarRating = ({ rating, setRating }) => {
  return (
    <div className="stars">
      {[1, 2, 3, 4, 5].map((star) => (
        <span
          key={star}
          className={star <= rating ? 'active' : ''}
          onClick={() => setRating(star)}
        >
          ★
        </span>
      ))}
    </div>
  );
};

// Static View Stars component
const ViewStars = ({ rating }) => {
  const safeRating = Math.max(0, Math.min(5, Number(rating) || 0));
  return (
    <div style={{ color: '#fbbf24'}}>
      {'★'.repeat(safeRating)}{'☆'.repeat(5 - safeRating)}
    </div>
  );
}

const App = () => {
  const [view, setView] = useState('submit'); // 'submit' or 'all'
  const [feedbacks, setFeedbacks] = useState([]);

  // Notification Message
  const [message, setMessage] = useState("");

  // Form State
  const [formData, setFormData] = useState({ name: "", course: "", rating: 5, feedback: "" });

  // Edit State
  const [editingId, setEditingId] = useState(null);
  const [editData, setEditData] = useState({});

  const fetchFeedbacks = async () => {
    try {
      const res = await axios.get('/api/feedbacks');
      setFeedbacks(res.data);
    } catch (err) {
      console.error(err);
    }
  }

```

```

    }
  };

  useEffect(() => {
    if (view === 'all') fetchFeedbacks();
  }, [view]);

  const showMessage = (msg) => {
    setMessage(msg);
    setTimeout(() => setMessage(""), 3000);
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      await axios.post('/api/feedbacks', formData);
      setFormData({ name: "", course: "", rating: 5, feedback: "" });
      showMessage('Feedback submitted successfully!');
      setView('all');
    } catch (err) {
      alert(err.response?.data?.error || 'Error submitting form');
    }
  };

  const handleDelete = async (id) => {
    if(confirm("Delete this feedback?")) {
      try {
        await axios.delete(`/api/feedbacks/${id}`);
        setFeedbacks(feedbacks.filter(f => f._id !== id));
        showMessage('Feedback deleted');
      } catch (err) {
        alert('Failed to delete');
      }
    }
  };

  const handleEditStart = (fb) => {
    setEditingId(fb._id);
    setEditData({ ...fb });
  };

  const handleEditSave = async () => {
    try {
      const res = await axios.put(`/api/feedbacks/${editingId}`, editData);
      setFeedbacks(feedbacks.map(f => f._id === editingId ? res.data : f));
      setEditingId(null);
      showMessage('Feedback updated');
    } catch (err) {

```

```

        alert('Update failed');
    }
};

return (
    <div className="container">
        <header className="header">
            <h1>Student Feedback</h1>
            <button
                className="btn btn-primary"
                onClick={() => setView(view === 'submit' ? 'all' : 'submit')}
            >
                {view === 'submit' ? 'View All Feedbacks' : 'Submit Feedback'}
            </button>
        </header>

```

Server.js

```

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const path = require('path');
require('dotenv').config();

const app = express();
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://localhost:27017/fsdl_a7';

// Middleware
app.use(cors());
app.use(express.json());

// Serve static elements from public directory
app.use(express.static(path.join(__dirname, 'public')));

// Set up MongoDB Mongoose Connection
mongoose.connect(MONGO_URI).then(() => {
    console.log('Connected to MongoDB database');
}).catch((error) => {
    console.error('Error connecting to MongoDB:', error.message);
});

// Import Feedback Model
const Feedback = require('./models/Feedback');

// API Routes
app.post('/api/feedbacks', async (req, res) => {
    try {

```



```

    const { name, course, rating, feedback } = req.body;
    if (!name || !course || !rating || !feedback) {
      return res.status(400).json({ error: 'All fields are required.' });
    }
    const newFeedback = new Feedback({ name, course, rating, feedback });
    const savedFeedback = await newFeedback.save();
    res.status(201).json(savedFeedback);
  } catch (error) {
    res.status(500).json({ error: 'Failed to create feedback', details: error.message });
  }
});

app.put('/api/feedbacks/:id', async (req, res) => {
  try {
    const { name, course, rating, feedback } = req.body;
    const updatedFeedback = await Feedback.findByIdAndUpdate(
      req.params.id,
      { name, course, rating, feedback },
      { new: true, runValidators: true }
    );
    if (!updatedFeedback) return res.status(404).json({ error: 'Feedback not found' });
    res.status(200).json(updatedFeedback);
  } catch (error) {
    res.status(500).json({ error: 'Failed to update feedback', details: error.message });
  }
});

app.delete('/api/feedbacks/:id', async (req, res) => {
  try {
    const deletedFeedback = await Feedback.findByIdAndDelete(req.params.id);
    if (!deletedFeedback) return res.status(404).json({ error: 'Feedback not found' });
    res.status(200).json({ message: 'Feedback deleted successfully' });
  } catch (error) {
    res.status(500).json({ error: 'Failed to delete feedback', details: error.message });
  }
});

// Fallback all routes to index.html to allow client side routing if necessary
app.use((req, res, next) => {
  if (req.method === 'GET' && !req.path.startsWith('/api')) {
    res.sendFile(path.join(__dirname, 'public', 'index.html'));
  } else {
    next();
  }
});

// Start Server
app.listen(PORT, () => {

```

```
console.log(`Server is running on http://localhost:${PORT}`);  
});
```

Student Feedback Screen

Student Feedback

[View All Feedbacks](#)

We value your feedback

First & Last Name

Course Name

Rating
★★★★★

Comments

[Submit Review](#)

View all Feedbacks/edit/delete

Student Feedback

[Submit Feedback](#)

Ayan Mujawar
Deep Learning
★★★★☆
"all concepts not covered"

[Edit](#) [Delete](#)

Om Mundada
UI-UX
★★★★★
"NA"

[Edit](#) [Delete](#)

Paritosh
Cloud Computing
★★★★☆
"Learned the basics and fundamentals of Cloud"

[Edit](#) [Delete](#)

Connection with MongoDB:

The screenshot displays the MongoDB Compass web interface. On the left sidebar, the 'feedbacks' collection is selected under the 'localhost:27017' database. The main panel shows a list of three documents. The first document is for 'Paritosh' in the 'Cloud Computing' course with a rating of 4. The second document is for 'Om Mundada' in the 'UI-UX' course with a rating of 5. The third document is for 'Ayan Mujawar' in the 'Deep Learning' course with a rating of 3. Each document includes a feedback comment, creation and update timestamps, and a version number.

```
{
  "_id": ObjectId("69e3de2940efa42e573d86ab"),
  "name": "Paritosh",
  "course": "Cloud Computing",
  "rating": 4,
  "feedback": "Learned the basics and fundamentals of Cloud",
  "createdAt": "2026-04-18T19:40:25.951+00:00",
  "updatedAt": "2026-04-18T19:40:25.951+00:00",
  "__v": 0
}
```

```
{
  "_id": ObjectId("69e488eb40efa42e573d86ac"),
  "name": "Om Mundada",
  "course": "UI-UX",
  "rating": 5,
  "feedback": "NA",
  "createdAt": "2026-04-19T07:48:59.977+00:00",
  "updatedAt": "2026-04-19T07:48:59.977+00:00",
  "__v": 0
}
```

```
{
  "_id": ObjectId("69e4892d40efa42e573d86ad"),
  "name": "Ayan Mujawar",
  "course": "Deep Learning",
  "rating": 3,
  "feedback": "all concepts not covered",
  "createdAt": "2026-04-19T07:50:05.816+00:00",
  "updatedAt": "2026-04-19T07:50:05.816+00:00",
  "__v": 0
}
```

Conclusion

The Student Feedback System demonstrates how modern web technologies like React.js, Express.js, and MongoDB can be integrated to build an efficient and scalable application. It simplifies feedback collection, improves data organization, and enhances user experience.